Using IntegratedLearner for Multi-omics Prediction and Classification

Himel Mallick, Nalin Arora and Anupreet Porwal

2026-03-10

This vignette is a practical tutorial for binary, continuous, and survival outcome workflows in `IntegratedLearner`.

The goal is to show a complete end-to-end pattern you can adapt to your own multi-omics study:

1. Build correctly formatted training/validation inputs.
2. Fit per-layer, stacked, and concatenated learners.
3. Interpret model outputs (AUC/R2, layer weights, and feature signals).

`IntegratedLearner` supports three integration paradigms:

- Early fusion: concatenated features across layers.
- Late fusion: layer-specific models combined by a meta-learner.
- Intermediate fusion: additional integration strategies for downstream analysis.

Load Packages

```r
# Main package
library(IntegratedLearner)

# Needed for BART-based sections
options(java.parameters = "-Xmx6G")

# Tutorial dependencies
library(tidyverse)
library(SuperLearner)
library(caret)
library(cowplot)
library(bayesplot)
library(bartMachine)
library(curatedMetagenomicData)
library(TreeSummarizedExperiment)
library(SummarizedExperiment)
library(MultiAssayExperiment)
library(mlr3)
library(mlr3proba)
library(mlr3pipelines)
library(mlr3extralearners)
library(survival)
library(S4Vectors)
```

Input Data Contract

For the `PCL_*` interface used in this tutorial, each dataset is a list with:

- `feature_table`: data frame with features in rows and samples in columns.
- `sample_metadata`: data frame with samples in rows. Must include:
  - `subjectID`: subject identifier (can repeat for longitudinal data).
  - `Y`: outcome (binary, continuous, or survival-encoded).
- `feature_metadata`: data frame with features in rows. Must include:
  - `featureID`: unique feature identifier.
  - `featureType`: layer label (for example, species, metabolites).

Required alignments:

- `rownames(feature_table) == rownames(feature_metadata)`
- `colnames(feature_table) == rownames(sample_metadata)`

If you provide a validation set, it must use the same feature set and ordering as training.

For survival workflows, include `time` and `event` columns in `sample_metadata` (with `event` coded as 0/1). You can also provide `Y` as a `Surv(time, event)` object.

Alternative Input Mode: MAE (Complete Binary Example)

`IntegratedLearner` accepts `MultiAssayExperiment` inputs through `MAE_train`/`MAE_valid`. This is often the cleanest path when each omics layer is already represented as a `SummarizedExperiment`/`TreeSummarizedExperiment`.

```r
# 1) Download two aligned layers from curatedMetagenomicData
asnicar_tax <- curatedMetagenomicData(
  "DavidLA_2015.relative_abundance",
  dryrun = FALSE
)[[1]]
#> 
#> $`2021-03-31.DavidLA_2015.relative_abundance`
#> dropping rows without rowTree matches:
#>   k__Bacteria|p__Actinobacteria|c__Coriobacteriia|o__Coriobacteriales|f__Coriobacteriaceae|g__Collinsel
#>   k__Bacteria|p__Actinobacteria|c__Coriobacteriia|o__Coriobacteriales|f__Coriobacteriaceae|g__Enorma|s_
#>   k__Bacteria|p__Firmicutes|c__Bacilli|o__Lactobacillales|f__Carnobacteriaceae|g__Granulicatella|s__Gra
#>   k__Bacteria|p__Proteobacteria|c__Betaproteobacteria|o__Burkholderiales|f__Sutterellaceae|g__Sutterell

asnicar_path <- curatedMetagenomicData(
  "DavidLA_2015.pathway_abundance",
  dryrun = FALSE
)[[1]]

tax_tse  <- as(asnicar_tax,  "TreeSummarizedExperiment")
path_tse <- as(asnicar_path, "TreeSummarizedExperiment")

# 2) Keep common samples in both layers
common_samples <- intersect(colnames(tax_tse), colnames(path_tse))
common_samples <- as.character(common_samples)

tax_tse  <- tax_tse[, common_samples]
path_tse <- path_tse[, common_samples]

# 3) Build binary outcome and subject IDs inside each experiment
Yvec <- ifelse(as.character(colData(tax_tse)$disease) == "healthy", 0L, 1L)

SummarizedExperiment::colData(tax_tse)$Y <- Yvec
SummarizedExperiment::colData(path_tse)$Y <- Yvec

SummarizedExperiment::colData(tax_tse)$subjectID <- common_samples
SummarizedExperiment::colData(path_tse)$subjectID <- common_samples

# 4) Build top-level MAE colData
cd <- S4Vectors::DataFrame(
  Y = as.integer(Yvec),
  subjectID = common_samples,
  row.names = common_samples
)

# 5) Build explicit sampleMap
smap <- S4Vectors::DataFrame(
  assay   = c(rep("taxonomy", length(common_samples)),
              rep("pathway",  length(common_samples))),
  primary = c(common_samples, common_samples),
  colname = c(common_samples, common_samples)
)

smap$assay   <- as.character(smap$assay)
smap$primary <- as.character(smap$primary)
smap$colname <- as.character(smap$colname)

# 6) Build MAE container
mae <- MultiAssayExperiment(
  experiments = ExperimentList(
    taxonomy = tax_tse,
    pathway  = path_tse
  ),
  colData = cd,
  sampleMap = smap
)

# 7) Stratified train/validation split
y <- MultiAssayExperiment::colData(mae)$Y
names(y) <- rownames(MultiAssayExperiment::colData(mae))

set.seed(1)

i0 <- which(y == 0)
i1 <- which(y == 1)

train0 <- sample(i0, floor(0.7 * length(i0)))
train1 <- sample(i1, floor(0.7 * length(i1)))

train_ids <- names(y)[sort(c(train0, train1))]
valid_ids <- setdiff(names(y), train_ids)

mae_train <- mae[, train_ids]
mae_valid <- mae[, valid_ids]

# 8) Fit IntegratedLearner in MAE mode
fit_mae_bin <- IntegratedLearner(
  MAE_train = mae_train,
  MAE_valid = mae_valid,
  experiment = c("taxonomy", "pathway"),
  assay.type = c("relative_abundance", "pathway_abundance"),
  folds = 3,
  base_learner = "SL.randomForest",
  meta_learner = "SL.nnls.auc",
  family = binomial(),
  verbose = TRUE
)
#> Running base model for layer  1 ...
#> Number of covariates in All is: 10379
#> CV SL.randomForest_All
#> Number of covariates in All is: 10379
#> CV SL.randomForest_All
#> Number of covariates in All is: 10379
#> CV SL.randomForest_All
#> Non-Negative least squares convergence: TRUE
#> full SL.randomForest_All
#> Running base model for layer  2 ...
#> Number of covariates in All is: 345
#> CV SL.randomForest_All
#> Number of covariates in All is: 345
#> CV SL.randomForest_All
#> Number of covariates in All is: 345
#> CV SL.randomForest_All
#> Non-Negative least squares convergence: TRUE
#> full SL.randomForest_All
#> Running stacked model...
#> Number of covariates in All is: 2
#> CV SL.nnls.auc_All
#> Number of covariates in All is: 2
#> CV SL.nnls.auc_All
#> Number of covariates in All is: 2
#> CV SL.nnls.auc_All
#> Non-Negative least squares convergence: TRUE
#> full SL.nnls.auc_All
#> Running concatenated model...
#> Number of covariates in All is: 10724
#> CV SL.randomForest_All
#> Number of covariates in All is: 10724
#> CV SL.randomForest_All
#> Number of covariates in All is: 10724
#> CV SL.randomForest_All
#> Non-Negative least squares convergence: TRUE
#> full SL.randomForest_All
#> Time for model fit : 0.288 minutes
#> ========================================
#> Model fit for individual layers: SL.randomForest
#> Model fit for stacked layer: SL.nnls.auc
#> Model fit for concatenated layer: SL.randomForest
#> ========================================
#> AUC metric for training data: 
#> Individual layers: 
#>  pathway taxonomy 
#>    0.968    0.929 
#> ======================
#> Stacked model:0.929
#> ======================
#> Concatenated model:0.968
#> ======================
#> ========================================
#> AUC metric for test data: 
#> Individual layers: 
#>  pathway taxonomy 
#>    0.972    1.000 
#> ======================
#> Stacked model:0.972
#> ======================
#> Concatenated model:0.972
#> ======================
#> ========================================
#> Weights for individual layers predictions in IntegratedLearner: 
#>  pathway taxonomy 
#>    0.979    0.021 
#> ========================================

# 10) Results
fit_mae_bin$AUC.train
#>      pathway     taxonomy      stacked concatenated 
#>        0.968        0.929        0.929        0.968 
fit_mae_bin$AUC.test
#>      pathway     taxonomy      stacked concatenated 
#>        0.972        1.000        0.972        0.972 
fit_mae_bin$weights
#>    pathway   taxonomy 
#> 0.97855301 0.02144699
```

The returned object is the same style as `PCL` mode, so downstream interpretation (`AUC.train`, `R2.train`, `weights`, `plot.learner`) is unchanged.

Parameter Reference (Conbin and Survival)

Common Wrapper Parameters (`IntegratedLearner`)

Parameter	Default	Applies to	Description
`MAE_train`, `MAE_valid`	`NULL`	all	MAE-mode inputs (training and optional validation).
`PCL_train`, `PCL_valid`	`NULL`	all	PCL-mode inputs (training and optional validation).
`experiment`	`NULL`	MAE mode	Selected MAE experiment names/indices; defaults to all experiments.
`assay.type`	`NULL`	MAE mode	Assay names per selected experiment.
`na.rm`	`FALSE`	all	Drop features with missing values after extraction/prep.
`folds`	`5`	all	Outer CV folds.
`seed`	`1234`	all	Reproducibility seed.
`base_learner`	`"SL.BART"`	all	Base learner. Use `SL.*` IDs for continuous/binary and explicitly set a supported `surv.*` ID for survival runs.
`verbose`	`FALSE`	all	Print progress.
`family`	`gaussian()`	all	Non-survival: `gaussian()`/`binomial()`. Survival is auto-detected from metadata or family.
`...`	—	all	Passed to the selected backend (`IL_conbin` or `ILsurv`).

Conbin-Specific Parameters (Continuous/Binary Path)

Parameter	Default	Description
`base_screener`	`"All"`	SuperLearner screener used with base learner.
`meta_learner`	`"SL.nnls.auc"`	Stacked meta learner for late fusion.
`run_stacked`	`TRUE`	Enables late-fusion stacked model.
`run_concat`	`TRUE`	Enables early-fusion concatenated model.
`print_learner`	`TRUE`	Prints fit summary.
`refit.stack`	`FALSE`	Refit stacked learner on full data for final predictions.

Survival-Specific Parameters (via `...`)

Parameter	Default	Description
`do_early_fusion`	`TRUE`	Train an early-fusion survival model on all features.
`weight_method`	`"IBS"` or `"COX"`	Late-fusion weighting objective.
`intermediate_learners`	`c("surv.coxph")`	Intermediate fusion learners on layer-risk features.
`t_vec`, `t_vec_probs`	`NULL`, quantiles	Time grid used in COX-style weighting summaries.
`layer_score`	`"sum"`	Aggregation of cumulative hazard increments (`sum`, `mean`, `l2`).
`weight_lambda`	`0.02`	Regularization strength for COX weighting optimizer.
`weight_penalty`	`"l2_to_uniform"` or `"entropy"`	Penalty used while learning survival late-fusion weights.
`weight_cap`	`1.0`	Optional cap on individual layer weights.
`optim_maxit_cox`	`4000`	Max iterations for COX weighting optimization.
`optim_maxit_ibs`	`300`	Max iterations for IBS weighting optimization.
`ibs_shrink_to_uniform`	`0`	Shrink IBS weights toward uniform blend.

Supported Models and Fusion Modules

Supported Models

Path	Supported base models
Continuous/Binary (`IL_conbin`)	Any `SuperLearner`-compatible `SL.*` learner available in your R session. Package wrappers include: `SL.BART`, `SL.LASSO`, `SL.enet`, `SL.glmnet2`, `SL.horseshoe`, `SL.mxBART` (plus standard SuperLearner learners such as `SL.glm`, `SL.randomForest`, etc.).
Survival (`ILsurv`)	Built-in survival learner IDs: `surv.rfsrc`, `surv.ranger`, `surv.coxboost`, `surv.bart`, `surv.coxph`, `surv.glmnet`.

Supported Fusion Outputs

Path	Single-layer	Early fusion	Late fusion	Intermediate fusion
Continuous/Binary	Yes	`run_concat = TRUE`	`run_stacked = TRUE` with `meta_learner`	Not a separate branch in `IL_conbin`
Survival	Yes (`train_out$single`)	`do_early_fusion = TRUE`	Weighted layer blending (`weight_method = "IBS"` or `"COX"`)	`intermediate_learners` on layer-risk summaries

Output Reference: What You Get and How to Access It

This section summarizes the outputs produced by each integration method and where to find weights/importance values.

Conbin Outputs (Binary/Continuous)

Method	What it returns	Where to access
Single-layer (per omics layer)	Layer-specific predictions and metrics	`fit$yhat.train[, layer_name]`, `fit$yhat.test[, layer_name]` (if validation), `fit$AUC.train` / `fit$AUC.test` (binomial), `fit$R2.train` / `fit$R2.test` (gaussian)
Early fusion (concatenated)	One model on all features concatenated	Enable with `run_concat = TRUE`; outputs in `fit$yhat.train[, "concatenated"]`, `fit$model_fits$model_concat`, `fit$SL_fits$SL_fit_concat`
Late fusion (stacked)	Meta-model over layer-level predictions	Enable with `run_stacked = TRUE`; outputs in `fit$yhat.train[, "stacked"]`, `fit$model_fits$model_stacked`, `fit$SL_fits$SL_fit_stacked`
Layer weights (stacked)	Relative contribution of each layer in late fusion	`fit$weights` (available when `meta_learner = "SL.nnls.auc"` and `run_stacked = TRUE`)

Survival Outputs (Single/Early/Late/Intermediate)

Method	Training outputs	Validation outputs
Single-layer	`fit$train_out$single$metrics`	`fit$valid_out$single$valid_cindex`, `fit$valid_out$single$valid_auc`
Early fusion	`fit$train_out$early$train_cindex`, `fit$train_out$early$train_auc`	`fit$valid_out$early$valid_cindex`, `fit$valid_out$early$valid_auc`
Late fusion	`fit$train_out$late$weights`, `fit$train_out$late$train_cindex`, `fit$train_out$late$train_auc`	`fit$valid_out$late$valid_cindex`, `fit$valid_out$late$valid_auc`
Intermediate fusion	`fit$train_out$intermediate` (per learner listed in `intermediate_learners`)	`fit$valid_out$intermediate`

Importance Outputs (Conbin and Survival)

Importance type	Where to access	Notes
Conbin signed global feature importance	`fit$feature_importance_signed`	Always returned for non-survival fits; named numeric vector sorted by effect magnitude/sign.
Conbin signed per-layer importance	`fit$feature_importance_signed_by_layer`	List split by `featureType`.
Survival early-fusion combined importance	`fit$train_out$early$combined_importance`	Available when `do_early_fusion = TRUE`.
Survival late-fusion combined importance	`fit$train_out$late$combined_importance`	Weighted signed importance; names are prefixed like `layer::feature`.
BART-specific layer importance (optional)	`bartMachine::investigate_var_importance(fit$model_fits$model_layers[[layer]], plot = FALSE)`	Only for BART-based conbin fits (`base_learner = "SL.BART"`).

Quick Access Snippets

```r
# ---- Conbin: weights + top features ----
fit$weights
head(fit$feature_importance_signed, 20)
names(fit$feature_importance_signed_by_layer)
head(fit$feature_importance_signed_by_layer[[1]], 20)

# ---- Survival: late-fusion weights + top combined features ----
fit_surv$train_out$late$weights
head(fit_surv$train_out$late$combined_importance, 20)

# ---- Survival: inspect all fusion branches ----
fit_surv$train_out$single
fit_surv$train_out$early
fit_surv$train_out$late
fit_surv$train_out$intermediate
```

Example 1: Binary Outcome (IBD Classification)

This section uses the PRISM dataset (Franzosa et al., 2019) for classifying IBD status. The binary target is coded in `sample_metadata$Y`.

Step 1: Load and Inspect Training and Validation Data

```r
# Training data
load(url("https://github.com/himelmallick/IntegratedLearner/blob/master/data/PRISM.RData?raw=true"))

feature_table <- pcl$feature_table
sample_metadata <- pcl$sample_metadata
feature_metadata <- pcl$feature_metadata
rm(pcl)

# Quick checks
head(feature_table[1:5, 1:5])
#>                                G35127      G35128      G35152       G36347
#> Granulicella_unclassified -0.05253649 -0.05127158 -0.06133085  0.004087447
#> Actinomyces_graevenitzii   1.04668500 -1.32629194 -1.51654615 -3.247989324
#> Actinomyces_johnsonii     -0.70327678 -0.41575776 -0.29326475 -0.314302395
#> Actinomyces_massiliensis  -0.56800952  0.14722899  0.05608884 -1.077235688
#> Actinomyces_naeslundii    -0.49546119 -0.15921604 -0.03146485 -0.354377267
#>                                 G36348
#> Granulicella_unclassified -0.006164066
#> Actinomyces_graevenitzii  -0.717183019
#> Actinomyces_johnsonii     -0.340485318
#> Actinomyces_massiliensis  -0.159240362
#> Actinomyces_naeslundii    -0.139758576
head(sample_metadata[1:5, ])
#>        Diagnosis dysbiosis_score Y subjectID
#> G35127        CD       0.9341287 1    G35127
#> G35128        CD       0.5962682 1    G35128
#> G35152        CD       0.9505732 1    G35152
#> G36347        CD       0.9966957 1    G36347
#> G36348        CD       0.8475403 1    G36348
head(feature_metadata[1:5, ])
#>                                           featureID featureType
#> Granulicella_unclassified Granulicella_unclassified     species
#> Actinomyces_graevenitzii   Actinomyces_graevenitzii     species
#> Actinomyces_johnsonii         Actinomyces_johnsonii     species
#> Actinomyces_massiliensis   Actinomyces_massiliensis     species
#> Actinomyces_naeslundii       Actinomyces_naeslundii     species

table(feature_metadata$featureType)
#> 
#> metabolites     species 
#>        8831         340 
table(sample_metadata$Y)
#> 
#>   0   1 
#>  34 121 

all(rownames(feature_table) == rownames(feature_metadata))
#> [1] TRUE
all(colnames(feature_table) == rownames(sample_metadata))
#> [1] TRUE

# Independent validation data
load(url("https://github.com/himelmallick/IntegratedLearner/blob/master/data/NLIBD.RData?raw=true"))
feature_table_valid <- pcl$feature_table
sample_metadata_valid <- pcl$sample_metadata
rm(pcl)

all(rownames(feature_table) == rownames(feature_table_valid))
#> [1] TRUE
all(colnames(feature_table_valid) == rownames(sample_metadata_valid))
#> [1] TRUE
```

Step 2: Build PCL Inputs

```r
PCL_train <- list(
  feature_table = feature_table,
  sample_metadata = sample_metadata,
  feature_metadata = feature_metadata
)

PCL_valid <- list(
  feature_table = feature_table_valid,
  sample_metadata = sample_metadata_valid,
  feature_metadata = feature_metadata
)
```

Step 3: Fit the Model

`IntegratedLearner` fits one model per layer (`base_learner`) and then combines layer-level predictions with a meta-learner (`meta_learner`).

```r
fit <- IntegratedLearner(
  PCL_train = PCL_train,
  PCL_valid = PCL_valid,
  folds = 5,
  base_learner = "SL.randomForest",
  meta_learner = "SL.nnls.auc",
  verbose = TRUE,
  family = binomial()
)
#> Running base model for layer  1 ...
#> Number of covariates in All is: 8831
#> CV SL.randomForest_All
#> Number of covariates in All is: 8831
#> CV SL.randomForest_All
#> Number of covariates in All is: 8831
#> CV SL.randomForest_All
#> Number of covariates in All is: 8831
#> CV SL.randomForest_All
#> Number of covariates in All is: 8831
#> CV SL.randomForest_All
#> Non-Negative least squares convergence: TRUE
#> full SL.randomForest_All
#> Running base model for layer  2 ...
#> Number of covariates in All is: 340
#> CV SL.randomForest_All
#> Number of covariates in All is: 340
#> CV SL.randomForest_All
#> Number of covariates in All is: 340
#> CV SL.randomForest_All
#> Number of covariates in All is: 340
#> CV SL.randomForest_All
#> Number of covariates in All is: 340
#> CV SL.randomForest_All
#> Non-Negative least squares convergence: TRUE
#> full SL.randomForest_All
#> Running stacked model...
#> Number of covariates in All is: 2
#> CV SL.nnls.auc_All
#> Number of covariates in All is: 2
#> CV SL.nnls.auc_All
#> Number of covariates in All is: 2
#> CV SL.nnls.auc_All
#> Number of covariates in All is: 2
#> CV SL.nnls.auc_All
#> Number of covariates in All is: 2
#> CV SL.nnls.auc_All
#> Non-Negative least squares convergence: TRUE
#> full SL.nnls.auc_All
#> Running concatenated model...
#> Number of covariates in All is: 9171
#> CV SL.randomForest_All
#> Number of covariates in All is: 9171
#> CV SL.randomForest_All
#> Number of covariates in All is: 9171
#> CV SL.randomForest_All
#> Number of covariates in All is: 9171
#> CV SL.randomForest_All
#> Number of covariates in All is: 9171
#> CV SL.randomForest_All
#> Non-Negative least squares convergence: TRUE
#> full SL.randomForest_All
#> Time for model fit : 3.115 minutes
#> ========================================
#> Model fit for individual layers: SL.randomForest
#> Model fit for stacked layer: SL.nnls.auc
#> Model fit for concatenated layer: SL.randomForest
#> ========================================
#> AUC metric for training data: 
#> Individual layers: 
#> metabolites     species 
#>       0.934       0.955 
#> ======================
#> Stacked model:0.951
#> ======================
#> Concatenated model:0.957
#> ======================
#> ========================================
#> AUC metric for test data: 
#> Individual layers: 
#> metabolites     species 
#>       0.782       0.879 
#> ======================
#> Stacked model:0.908
#> ======================
#> Concatenated model:0.895
#> ======================
#> ========================================
#> Weights for individual layers predictions in IntegratedLearner: 
#> metabolites     species 
#>        0.18        0.82 
#> ========================================
```

Step 4: Inspect and Interpret Outputs

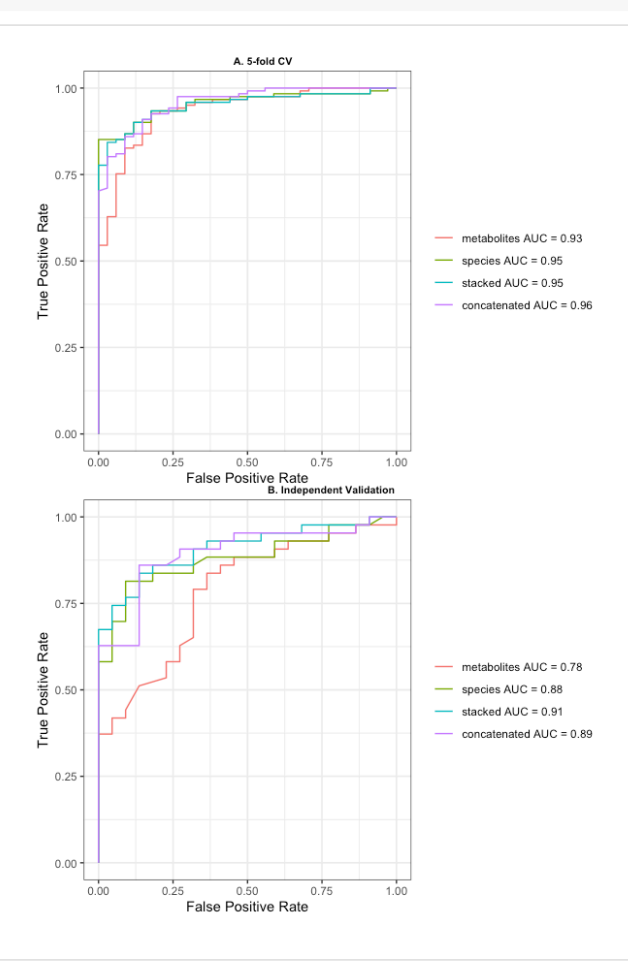
Core outputs for binary tasks include:

- `fit$AUC.train` and `fit$AUC.test`: AUC per layer and fusion model.
- `fit$weights`: layer contributions in the stacked model (when `SL.nnls.auc` is used).
- `fit$yhat.train` and `fit$yhat.test`: predicted probabilities.

```r
fit$AUC.train
#>  metabolites      species      stacked concatenated 
#>        0.934        0.955        0.951        0.957 
fit$AUC.test
#>  metabolites      species      stacked concatenated 
#>        0.782        0.879        0.908        0.895 
fit$weights
#> metabolites     species 
#>   0.1801901   0.8198099
```

Plot ROC summaries for train and validation sets:

```r
plot.obj <- IntegratedLearner:::plot.learner(fit)
```



In this PRISM setting, you can compare which single layer is strongest and whether stacked fusion outperforms both individual layers and simple concatenation.

Example 2: Continuous Outcome (Gestational Age)

This section uses the pregnancy dataset (Ghaemi et al., 2019), where `Y` is continuous gestational age.

Step 1: Load and Inspect Data

```r
load(url("https://github.com/himelmallick/IntegratedLearner/blob/master/data/pregnancy.RData?raw=true"))

feature_table <- pcl$feature_table
sample_metadata <- pcl$sample_metadata
feature_metadata <- pcl$feature_metadata
rm(pcl)

head(feature_table[1:5, 1:5])
#>        PTLG002_1 PTLG003_1  PTLG004_1 PTLG005_1 PTLG007_1
#> CEP135  28.21785  54.56723  53.776824  15.26909  11.04831
#> MIIP    10.10756  17.11006   4.336841   0.00000  19.88695
#> GNL3    45.25968  58.26670  56.378929  70.23788  64.08018
#> CEP70   79.09550  67.97782  93.675759 128.26033  66.28985
#> TIMP1  172.23675 121.62018 183.014677 247.35921 304.93329
head(sample_metadata[1:5, ])
#>            Y subjectID
#> PTLG002_1 11   PTLG002
#> PTLG003_1 11   PTLG003
#> PTLG004_1 11   PTLG004
#> PTLG005_1 11   PTLG005
#> PTLG007_1 11   PTLG007
head(feature_metadata[1:5, ])
#>        featureID featureType
#> CEP135    CEP135 CellfreeRNA
#> MIIP        MIIP CellfreeRNA
#> GNL3        GNL3 CellfreeRNA
#> CEP70      CEP70 CellfreeRNA
#> TIMP1      TIMP1 CellfreeRNA

table(feature_metadata$featureType)
#> 
#>     CellfreeRNA    ImmuneSystem    Metabolomics      Microbiome   PlasmaLuminex 
#>            9084             264             253             259              31 
#> PlasmaSomalogic    SerumLuminex 
#>             650              31 
length(unique(sample_metadata$subjectID))
#> [1] 17

all(rownames(feature_table) == rownames(feature_metadata))
#> [1] TRUE
all(colnames(feature_table) == rownames(sample_metadata))
#> [1] TRUE

# Optional speed-up for local experimentation
# top_n <- 50
# subsetIDs <- c(1:top_n, (nrow(feature_table) - top_n + 1):nrow(feature_table))
# feature_table <- feature_table[subsetIDs, ]
# feature_metadata <- feature_metadata[subsetIDs, ]
```

Step 2: Build PCL Input

```r
PCL_train <- list(
  feature_table = feature_table,
  sample_metadata = sample_metadata,
  feature_metadata = feature_metadata
)
```

Step 3: Fit Continuous Model

For this example, we use BART base learners (`SL.BART`) with leave-one-subject-out style CV (`folds = 17`).

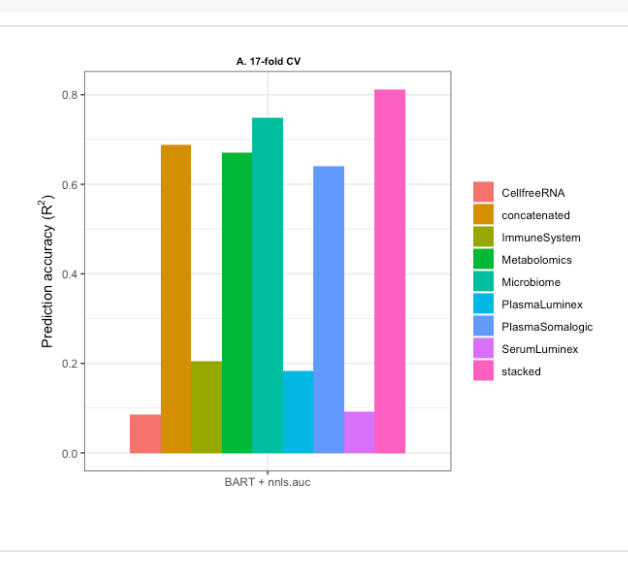
```r
fit <- IntegratedLearner(
  PCL_train = PCL_train,
  folds = 17,
  base_learner = "SL.BART",
  meta_learner = "SL.nnls.auc",
  family = gaussian()
)
#> Running base model for layer  1 ...
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> Running base model for layer  2 ...
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> Running base model for layer  3 ...
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> Running base model for layer  4 ...
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> Running base model for layer  5 ...
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> Running base model for layer  6 ...
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> Running base model for layer  7 ...
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> Running stacked model...
#> Running concatenated model...
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> serializing in order to be saved for future R sessions...done
#> Time for model fit : 5.393 minutes
#> ========================================
#> Model fit for individual layers: SL.BART
#> Model fit for stacked layer: SL.nnls.auc
#> Model fit for concatenated layer: SL.BART
#> ========================================
#> R^2 for training data: 
#> Individual layers: 
#>     CellfreeRNA    ImmuneSystem    Metabolomics      Microbiome   PlasmaLuminex 
#>      0.08548959      0.20491045      0.67114346      0.74893855      0.18314107 
#> PlasmaSomalogic    SerumLuminex 
#>      0.63949095      0.09200745 
#> ======================
#> Stacked model:0.8120559
#> ======================
#> Concatenated model:0.6880553
#> ======================
#> ========================================
#> Weights for individual layers predictions in IntegratedLearner: 
#>     CellfreeRNA    ImmuneSystem    Metabolomics      Microbiome   PlasmaLuminex 
#>           0.000           0.000           0.238           0.588           0.000 
#> PlasmaSomalogic    SerumLuminex 
#>           0.174           0.000 
#> ========================================
```

Step 4: Evaluate Predictive Accuracy

For continuous outcomes, `IntegratedLearner` reports `R2.train` (and `R2.test` if validation is provided).

```r
fit$R2.train
#>     CellfreeRNA    ImmuneSystem    Metabolomics      Microbiome   PlasmaLuminex 
#>      0.08548959      0.20491045      0.67114346      0.74893855      0.18314107 
#> PlasmaSomalogic    SerumLuminex         stacked    concatenated 
#>      0.63949095      0.09200745      0.81205589      0.68880530
```

```r
plot.obj <- IntegratedLearner:::plot.learner(fit)
```



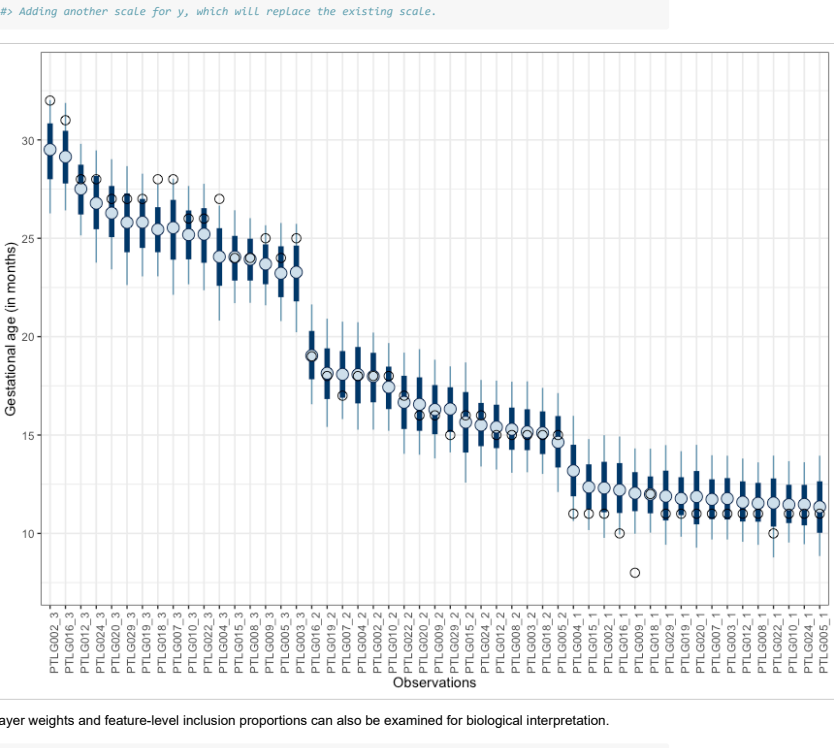
Step 5: Uncertainty and Feature-Level Interpretation (BART)

When using `SL.BART`, you can inspect posterior predictive distributions and derive weighted posterior summaries.

```r
weights <- fit$weights

dataX <- fit$X_train_layers
dataY <- fit$Y_train

post.samples <- vector("list", length(weights))
names(post.samples) <- names(dataX)

for (i in seq_along(post.samples)) {
  post.samples[[i]] <- bart_machine_get_posterior(
    fit$model_fits$model_layers[[i]],
    dataX[[i]]
  )$y_hat_posterior_samples
}

weighted.post.samples <- Reduce("+", Map("*", post.samples, weights))
rownames(weighted.post.samples) <- rownames(dataX[[1]])
names(dataY) <- rownames(dataX[[1]])
```

Visualize 68% and 95% credible intervals for observations:

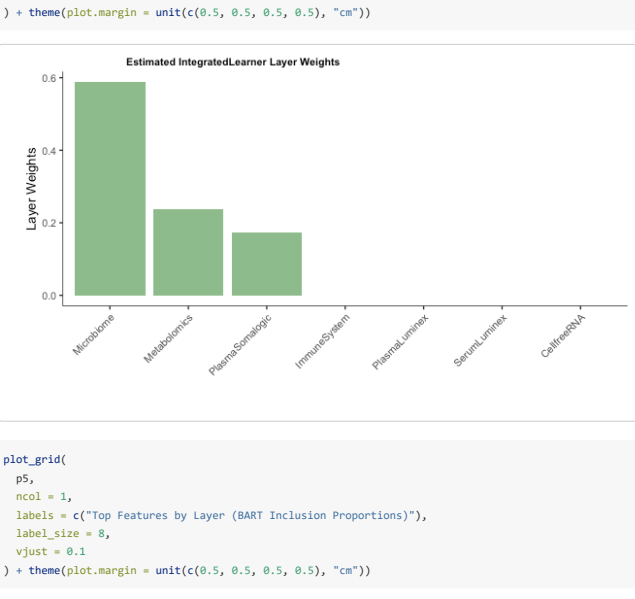
```r
ord_names <- names(sort(rowMeans(weighted.post.samples), decreasing = TRUE))

mcmc_intervals(t(weighted.post.samples), prob = 0.68, prob_outer = 0.95) +
  scale_y_discrete(limits = ord_names) +
  geom_point(aes(x = dataY[ord_names], y = ord_names), shape = 1, size = 3, color = "black") +
  coord_flip() +
  theme_bw() +
  labs(
    x = "Gestational age (in months)",
    y = "Observations"
  ) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
#> Scale for y is already present.
#> Adding another scale for y, which will replace the existing scale.
```



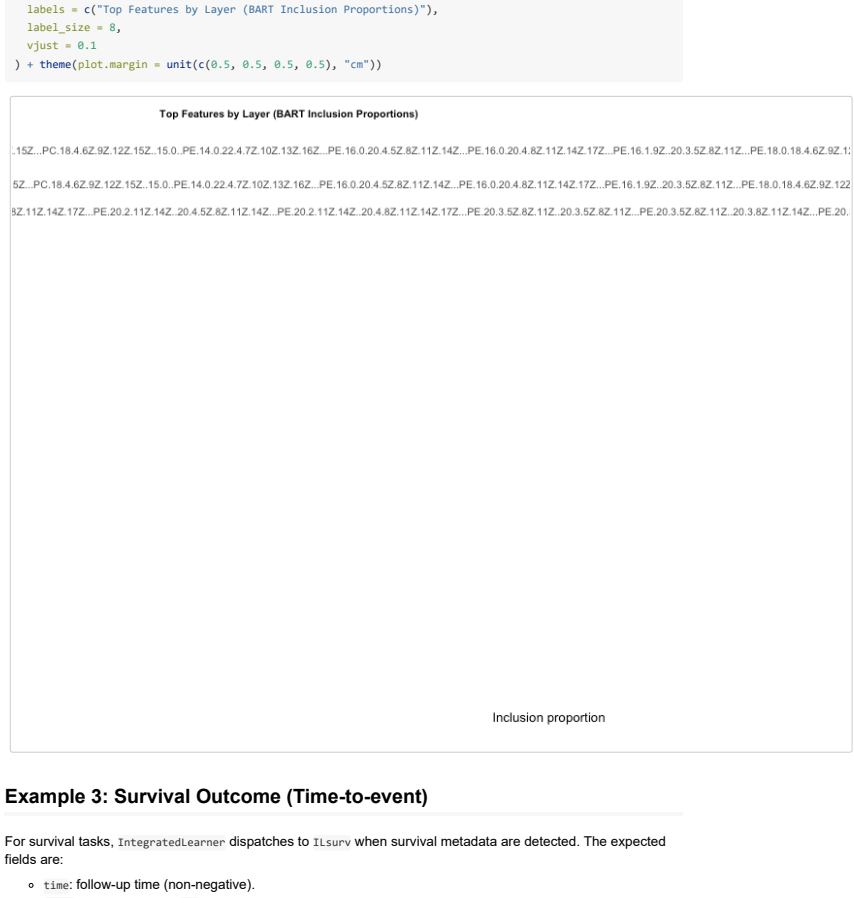
Layer weights and feature-level inclusion proportions can also be examined for biological interpretation.

```r
omicsEye_theme <- function() {
  angle <- 45
  hjust <- 1
  ggplot2::theme_bw() +
    ggplot2::theme(
      axis.text.x = ggplot2::element_text(size = 8, vjust = 1, hjust = hjust, angle = angle),
      axis.text.y = ggplot2::element_text(size = 8, hjust = 1),
      axis.title = ggplot2::element_text(size = 10),
      plot.title = ggplot2::element_text(size = 10),
      plot.subtitle = ggplot2::element_text(size = 8),
      legend.title = ggplot2::element_text(size = 6, face = "bold"),
      legend.text = ggplot2::element_text(size = 7),
      axis.line = ggplot2::element_line(colour = "black", linewidth = 0.25),
      axis.line.x = ggplot2::element_line(colour = "black", linewidth = 0.25),
      axis.line.y = ggplot2::element_line(colour = "black", linewidth = 0.25),
      panel.border = ggplot2::element_blank(),
      panel.grid.major = ggplot2::element_blank(),
      panel.grid.minor = ggplot2::element_blank()
    )
}

safe_var_importance <- function(model, layer_label) {
  tryCatch({
    qq <- bartMachine::investigate_var_importance(model, plot = FALSE)
    df <- cbind.data.frame(qq$avg_var_props, qq$sd_var_props)
    colnames(df) <- c("mean", "sd")
    df$type <- layer_label
    df
  }, error = function(e) {
    warning(sprintf("Skipping variable importance for %s: %s", layer_label, conditionMessage(e)))
    data.frame(mean = numeric(), sd = numeric(), type = character())
  })
}

vimp_stack <- cbind.data.frame(fit$weights)
colnames(vimp_stack) <- "mean"
vimp_stack$sd <- NA
vimp_stack$type <- "stack"

layer_names <- names(fit$model_fits$model_layers)
vimp_layers <- lapply(layer_names, function(layer_nm) {
  safe_var_importance(fit$model_fits$model_layers[[layer_nm]], layer_nm)
})
#> .....
#> .....
#> .....
#> .....
#> .....
#> .....
#> .....

vimp_layers <- vimp_layers[lengths(vimp_layers) > 0]
vimp_top <- do.call(
  rbind,
  lapply(vimp_layers, function(df) head(df[order(-df$mean), , drop = FALSE], 20))
)

VIMP <- as.data.frame(rbind.data.frame(vimp_stack, vimp_top))
VIMP <- tibble::rownames_to_column(VIMP, "ID")

p4 <- VIMP %>%
  dplyr::filter(type == "stack") %>%
  dplyr::arrange(desc(mean)) %>%
  ggplot(aes(y = mean, x = reorder(ID, -mean))) +
  geom_bar(stat = "identity", fill = "darkseagreen") +
  theme_bw() +
  omicsEye_theme() +
  ylab("Layer Weights") +
  xlab("")

p5 <- VIMP %>%
  dplyr::filter(type != "stack") %>%
  dplyr::arrange(mean) %>%
  dplyr::mutate(ID = stringr::str_replace_all(ID, stringr::fixed("_"), " ")) %>%
  ggplot(aes(reorder(ID, -mean), mean, fill = type)) +
  facet_wrap(. ~ type, scales = "free") +
  geom_bar(stat = "identity", fill = "lightsalmon") +
  geom_errorbar(aes(ymin = ifelse(mean - sd > 0, mean - sd, 0), ymax = mean + sd),
                width = 0.2,
                position = position_dodge(0.9)) +
  theme_bw() +
  coord_flip() +
  omicsEye_theme() +
  theme(strip.background = element_blank()) +
  ylab("Inclusion proportion") +
  xlab("")
```

```r
plot_grid(
  p4,
  ncol = 1,
  labels = c("Estimated IntegratedLearner Layer Weights"),
  label_size = 8,
  vjust = 0.1
) + theme(plot.margin = unit(c(0.5, 0.5, 0.5, 0.5), "cm"))
```



```r
plot_grid(
  p5,
  ncol = 1,
  labels = c("Top Features by Layer (BART Inclusion Proportions)"),
  label_size = 8,
  vjust = 0.1
) + theme(plot.margin = unit(c(0.5, 0.5, 0.5, 0.5), "cm"))
```



Example 3: Survival Outcome (Time-to-event)

For survival tasks, `IntegratedLearner` dispatches to `ILsurv` when survival metadata are detected. The expected fields are:

- `time`: follow-up time (non-negative).
- `event`: event indicator (`0/1`).
- optional `Y`: `Surv(time, event)` convenience column.

This section provides a complete MAE workflow, followed by an equivalent PCL sketch.

Step 1: Construct MAE Survival Inputs from TCGA_BRCA

```r
load(url("https://github.com/himelmallick/IntegratedLearner/blob/master/data/Survival/TCGA_BRCA.RData?raw=
# Objects loaded: gene_all, mir_all

# Keep deterministic ordering
gene_all <- gene_all[order(gene_all$patient_id), ]
mir_all  <- mir_all[order(mir_all$patient_id), ]
rownames(gene_all) <- gene_all$patient_id
rownames(mir_all)  <- mir_all$patient_id

# Sample metadata
sample_ids <- rownames(gene_all)
sample_metadata <- data.frame(
  subjectID = sample_ids,
  time = gene_all$OS.time,
  event = gene_all$OS,
  Y = Surv(gene_all$OS.time, gene_all$OS),
  row.names = sample_ids
)

# Feature matrices (features in rows)
drop_cols <- c("patient_id", "age", "stage_i", "stage_ii", "race_white", "OS.time", "OS")
mat_gene <- t(as.data.frame(gene_all[, setdiff(names(gene_all), drop_cols)]))
mat_mirna <- t(as.data.frame(mir_all[, setdiff(names(mir_all), drop_cols)]))

# Per-layer SummarizedExperiments
se_gene <- SummarizedExperiment(
  assays = list(relative_abundance = mat_gene),
  colData = S4Vectors::DataFrame(sample_metadata[colnames(mat_gene), ])
)

se_mirna <- SummarizedExperiment(
  assays = list(pathway_abundance = mat_mirna),
  colData = S4Vectors::DataFrame(sample_metadata[colnames(mat_mirna), ])
)

mae <- MultiAssayExperiment(
  experiments = list(taxonomy = se_gene, pathway = se_mirna),
  colData = S4Vectors::DataFrame(sample_metadata)
)
```

Step 2: Split MAE into Training and Validation

```r
set.seed(1)
idx <- sample(seq_along(sample_ids), size = floor(0.7 * length(sample_ids)))
mae_train <- mae[, idx, ]
mae_valid <- mae[, -idx, ]

# Utility to keep Y/time/event present in every experiment colData after subsetting
propagate_Y <- function(mae_obj) {
  exps <- MultiAssayExperiment::experiments(mae_obj)
  for (nm in names(exps)) {
    se <- exps[[nm]]
    if (!"Y" %in% colnames(SummarizedExperiment::colData(se))) {
      cd <- as.data.frame(MultiAssayExperiment::colData(mae_obj))[colnames(se), , drop = FALSE]
      SummarizedExperiment::colData(se) <- S4Vectors::DataFrame(cd)
    }
    exps[[nm]] <- se
  }
  MultiAssayExperiment::experiments(mae_obj) <- exps
  mae_obj
}

mae_train <- propagate_Y(mae_train)
mae_valid <- propagate_Y(mae_valid)
```

Step 3: Fit Survival Model (MAE Mode)

```r
fit_surv_mae <- IntegratedLearner(
  MAE_train = mae_train,
  MAE_valid = mae_valid,
  experiment = c("taxonomy", "pathway"),
  assay.type = c("relative_abundance", "pathway_abundance"),
  folds = 2,
  base_learner = "surv.coxboost",  # alternatives: surv.coxph, surv.ranger, surv.glmnet, ...
  weight_method = "COX",           # alternative: "IBS"
  verbose = TRUE
)
#> Found 2 layer(s): taxonomy, pathway
#> Processing layer: taxonomy
#> Processing layer: pathway
#> 
#> Chosen method: COX
#> Weights:
#>  taxonomy   pathway 
#> 0.4679153 0.5320847 
#> Train C-index: 0.6575
```

Step 4: Interpret Survival Outputs

```r
# Single-layer training metrics
# fit_surv_mae$train_out$single$metrics

# Late fusion (weighted integration)
fit_surv_mae$train_out$late$weights
#>  taxonomy   pathway 
#> 0.4679153 0.5320847 
fit_surv_mae$train_out$late$train_cindex
#> [1] 0.6574672
fit_surv_mae$train_out$late$train_auc
#>             time       AUC
#> t=512     512.00 0.6167299
#> t=972.5   972.50 0.6998332
#> t=1878.75 1878.75 0.6591858

# Early and intermediate fusion summaries
#fit_surv_mae$train_out$early
#fit_surv_mae$train_out$intermediate

# Validation metrics
fit_surv_mae$valid_out$late$valid_cindex
#> [1] 0.6534671
fit_surv_mae$valid_out$late$valid_auc
#>             time       AUC
#> t=512     512.00 0.8069608
#> t=972.5   972.50 0.6998237
#> t=1878.75 1878.75 0.6373053
fit_surv_mae$valid_out$single$valid_cindex
#> $taxonomy
#> [1] 0.6835693
#> 
#> $pathway
#> [1] 0.6151227
```

Equivalent PCL Survival Pattern (Same Data)

```r
# Convert to PCL-style objects if preferred:
feature_table <- as.data.frame(rbind(mat_gene, mat_mirna))
feature_metadata <- rbind(
  data.frame(featureID = rownames(mat_gene), featureType = "gene", row.names = rownames(mat_gene)),
  data.frame(featureID = rownames(mat_mirna), featureType = "mirna", row.names = rownames(mat_mirna))
)

PCL_train <- list(
  feature_table = feature_table[, rownames(sample_metadata)[idx], drop = FALSE],
  sample_metadata = sample_metadata[rownames(sample_metadata)[idx], , drop = FALSE],
  feature_metadata = feature_metadata
)

PCL_valid <- list(
  feature_table = feature_table[, rownames(sample_metadata)[-idx], drop = FALSE],
  sample_metadata = sample_metadata[rownames(sample_metadata)[-idx], , drop = FALSE],
  feature_metadata = feature_metadata
)

fit_surv_pcl <- IntegratedLearner(
  PCL_train = PCL_train,
  PCL_valid = PCL_valid,
  folds = 2,
  base_learner = "surv.coxboost",
  weight_method = "IBS"
)
```

Session Information

```r
sessionInfo()
#> R version 4.5.1 (2025-06-13)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS Sonoma 14.2.1
#> 
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib 
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK vers
#> 
#> locale:
#> [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
#> 
#> time zone: America/New_York
#> tzcode source: internal
#> 
#> attached base packages:
#> [1] stats4    splines   stats     graphics  grDevices utils     datasets 
#> [8] methods   base     
#> 
#> other attached packages:
#>  [1] prodlim_2025.04.28              Matrix_1.7-4                   
#>  [3] future_1.69.0                   survival_3.8-3                 
#>  [5] mlr3extralearners_1.3.1         mlr3pipelines_0.11.0           
#>  [7] mlr3proba_0.8.5                 mlr3_1.5.0                     
#>  [9] MultiAssayExperiment_1.36.0     curatedMetagenomicData_3.16.1  
#> [11] TreeSummarizedExperiment_2.16.1 Biostrings_2.76.0              
#> [13] GenomeInfoDb_1.44.3             XVector_0.50.0                 
#> [15] SingleCellExperiment_1.30.1     SummarizedExperiment_1.40.0    
#> [17] Biobase_2.70.0                  GenomicRanges_1.62.0           
#> [19] Seqinfo_1.0.0                   IRanges_2.44.0                 
#> [21] S4Vectors_0.48.0                BiocGenerics_0.56.0            
#> [23] generics_0.1.4                  MatrixGenerics_1.22.0          
#> [25] matrixStats_1.5.0               bartMachine_1.3.4.1            
#> [27] missForest_1.6.1                randomForest_4.7-1.2           
#> [29] bartMachineJARs_1.2.1           rJava_1.0-11                   
#> [31] bayesplot_1.14.0                cowplot_1.2.0                  
#> [33] caret_7.0-1                     lattice_0.22-7                 
#> [35] SuperLearner_2.0-40             gam_1.22-7                     
#> [37] foreach_1.5.2                   nnls_1.6                       
#> [39] lubridate_1.9.5                 forcats_1.0.1                  
#> [41] stringr_1.6.0                   dplyr_1.2.0                    
#> [43] purrr_1.2.1                     readr_2.1.5                    
#> [45] tidyr_1.3.2                     tibble_3.3.1                   
#> [47] ggplot2_4.0.2                   tidyverse_2.0.0                
#> [49] IntegratedLearner_0.1.1        
#> 
#> loaded via a namespace (and not attached):
#>   [1] ggtext_0.1.2                fs_1.6.6                   
#>   [3] CoxBoost_1.5                DirichletMultinomial_1.50.0
#>   [5] httr_1.4.7                  RColorBrewer_1.1-3         
#>   [7] numDeriv_2016.8-1.1         backports_1.5.0            
#>   [9] tools_4.5.1                 doRNG_1.8.6.2              
#>  [11] vegan_2.7-2                 R6_2.6.1                   
#>  [13] mgcv_1.9-3                  lazyeval_0.2.2             
#>  [15] permute_0.9-8               withr_3.0.2                
#>  [17] gridExtra_2.3               textshaping_1.0.4          
#>  [19] cli_3.6.5                   sandwich_3.1-1             
#>  [21] labeling_0.4.3              slam_0.1-55                
#>  [23] sass_0.4.10                 mvtnorm_1.3-3              
#>  [25] S7_0.2.1                    systemfonts_1.3.1          
#>  [27] yulab.utils_0.2.1           survdistr_0.0.1            
#>  [29] set6_0.2.6                  paradox_1.0.1              
#>  [31] scater_1.36.0               decontam_1.28.0            
#>  [33] parallelly_1.46.1           itertools_0.1-3            
#>  [35] readxl_1.4.5                fillpattern_1.0.2          
#>  [37] rstudioapi_0.17.1           RSQLite_2.4.3              
#>  [39] distributional_0.5.0        rbiom_2.2.1                
#>  [41] ggbeeswarm_0.7.2            DECIPHER_3.4.0             
#>  [43] abind_1.4-8                 lifecycle_1.0.5            
#>  [45] multcomp_1.4-29             yaml_2.3.12                
#>  [47] recipes_1.3.1               SparseArray_1.10.1         
#>  [49] BiocFileCache_2.16.2        grid_4.5.1                 
#>  [51] blob_1.2.4                  ExperimentHub_2.16.1       
#>  [53] crayon_1.5.3                beachmat_2.24.0            
#>  [55] KEGGREST_1.48.1             pillar_1.11.1              
#>  [57] knitr_1.51                  estimability_1.5.1         
#>  [59] future.apply_1.20.2         codetools_0.2-20           
#>  [61] glue_1.8.0                  data.table_1.18.2.1        
#>  [63] vctrs_0.7.1                 png_0.1-8                  
#>  [65] treeio_1.32.0               Rdpack_2.6.4               
#>  [67] distr6_1.8.4                cellranger_1.1.0           
#>  [69] gtable_0.3.6                cachem_1.1.0               
#>  [71] gower_1.0.2                 xfun_0.55                  
#>  [73] mime_0.13                   rbibutils_2.4              
#>  [75] S4Arrays_1.10.0             timeDate_4052.112          
#>  [77] iterators_1.0.14            hardhat_1.4.2              
#>  [79] lava_1.8.2                  bluster_1.18.0             
#>  [81] TH.data_1.1-5               ROCR_1.0-12                
#>  [83] ipred_0.9-15                nlme_3.1-168               
#>  [85] bit64_4.6.0-1               filelock_1.0.3             
#>  [87] tensorA_0.36.2.1            bslib_0.9.0                
#>  [89] irlba_2.3.5.1               vipor_0.4.7                
#>  [91] otel_0.2.0                  rpart_4.1.24               
#>  [93] DBI_1.2.3                   nnet_7.3-20                
#>  [95] mlr3misc_0.21.0             tidyselect_1.2.1           
#>  [97] emmeans_2.0.0               bit_4.6.0                  
#>  [99] compiler_4.5.1              curl_7.0.0                 
#> [101] ooplah_0.2.0                BiocNeighbors_2.2.0        
#> [103] lgr_0.5.2                   timeROC_0.4                
#> [105] xml2_1.4.1                  DelayedArray_0.36.0        
#> [107] posterior_1.6.1             checkmate_2.3.4            
#> [109] scales_1.4.0                pec_2025.06.24             
#> [111] quadprog_1.5-8              palmerpenguins_0.1.1       
#> [113] rappdirs_0.3.3              digest_0.6.39              
#> [115] param6_0.2.4                rmarkdown_2.30             
#> [117] htmltools_0.5.9             pkgconfig_2.0.3            
#> [119] sparseMatrixStats_1.20.0    dbplyr_2.5.1               
#> [121] fastmap_1.2.0               rlang_1.1.7                
#> [123] UCSC.utils_1.4.0            DelayedMatrixStats_1.30.0  
#> [125] farver_2.1.2                jquerylib_0.1.4            
#> [127] zoo_1.8-15                  jsonlite_2.0.0             
#> [129] BiocParallel_1.42.2         ModelMetrics_1.2.2.2       
#> [131] BiocSingular_1.24.0         magrittr_2.0.4             
#> [133] scuttle_1.18.0              GenomeInfoDbData_1.2.14    
#> [135] patchwork_1.3.2             Rcpp_1.1.1                 
#> [137] ape_5.8-1                   ggnewscale_0.5.2           
#> [139] viridis_0.6.5               stringi_1.8.7              
#> [141] pROC_1.19.0.1               MASS_7.3-65                
#> [143] AnnotationHub_3.16.1        plyr_1.8.9                 
#> [145] mlr3cmprsk_0.0.1            dictionar6_0.1.3           
#> [147] parallel_4.5.1              listenv_0.10.0             
#> [149] ggrepel_0.9.6               gridtext_0.1.5             
#> [151] hms_1.1.4                   timereg_2.0.7              
#> [153] uuid_1.2-2                  igraph_2.2.1               
#> [155] ranger_0.18.0               rngtools_1.5.2             
#> [157] reshape2_1.4.5              ScaledMatrix_1.16.0        
#> [159] BiocVersion_3.21.1          evaluate_1.0.5             
#> [161] BiocManager_1.30.26         nloptr_2.2.1               
#> [163] tzdb_0.5.0                  BiocBaseUtils_1.12.0       
#> [165] rsvd_1.0.5                  xtable_1.8-4               
#> [167] tidytree_0.4.6              ragg_1.5.0                 
#> [169] viridisLite_0.4.3           class_7.3-23               
#> [171] memoise_2.0.1               beeswarm_0.4.0             
#> [173] AnnotationDbi_1.70.0        cluster_2.1.8.1            
#> [175] timechange_0.4.0            globals_0.19.0             
#> [177] mia_1.16.1
```

References

Franzosa EA et al. (2019). Gut microbiome structure and metabolic activity in inflammatory bowel disease. *Nature Microbiology* 4(2):293-305.

Ghaemi MS et al. (2019). Multiomics modeling of the immunome, transcriptome, microbiome, proteome and metabolome adaptations during human pregnancy. *Bioinformatics* 35(1):95-103.

Citation

Mallick et al. (2024). An integrated Bayesian framework for multi-omics prediction and classification. *Statistics in Medicine* 43(5):983-1002.